VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY

INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Data Science and Data Visualization Lab 02

Student name:

Pham Nhu Nguyen Trung - ITCSIU24091

Nguyen Quynh Anh - ITCSIU24005

7. Think

7.1

1. How many rect elements will be created?
   3 rect elements will be created.
2. What value is bound to the first rect? **10**
3. What value is bound to the third rect? **30**
4. What does d represent in: .attr("height", function(d) { return d; })
   d represent to the number or data which could be set to the attribute
   e.g : dataset [10,20,30]
   functions will run 3 times
   1st: d=10 -> height =10
   2nd: d=20 -> h=20
   3rd: d=30 -> h=30
5. What would happen if the dataset contained 5 values?
   Draw 5 rectangles with 5 values in the dataset

7.2

Dataset → select() → selectAll(“rect”) → data() → enter() → append(“rect”) → visual elements

The explanation for all the following roles:

- selectAll()
  - It will return all matched elements
- data()
  - This is the method to bind data to DOM elements
- enter()
  - Create a new placeholder element, or it can be said it is used to create new, data-bound elements.
- append()
  - In this case, append(“rect”) will create a new rectangle in the selected part above.

7.3 Trace d

The value of d would be in this order: 10, 60, 30, 80. D3 can use the same for all four elements because it automatically iterates through all values of d.

7.4 Predict visual encoding

The answer: **10 60 30 80**

The mapping rule is that whichever element is bigger than 50 will be colored red.

7.5 CSV Reasoning

1. What type might Midterm have after CSV loading? **string**
2. What type might Final have? **string**
3. Why could this matter when calculating: 0.4 × Midterm + 0.6 × Final.

We need to consider the event that Midterm and Final are string instead of integer or float types.

4. What should a row-conversion function accomplish?

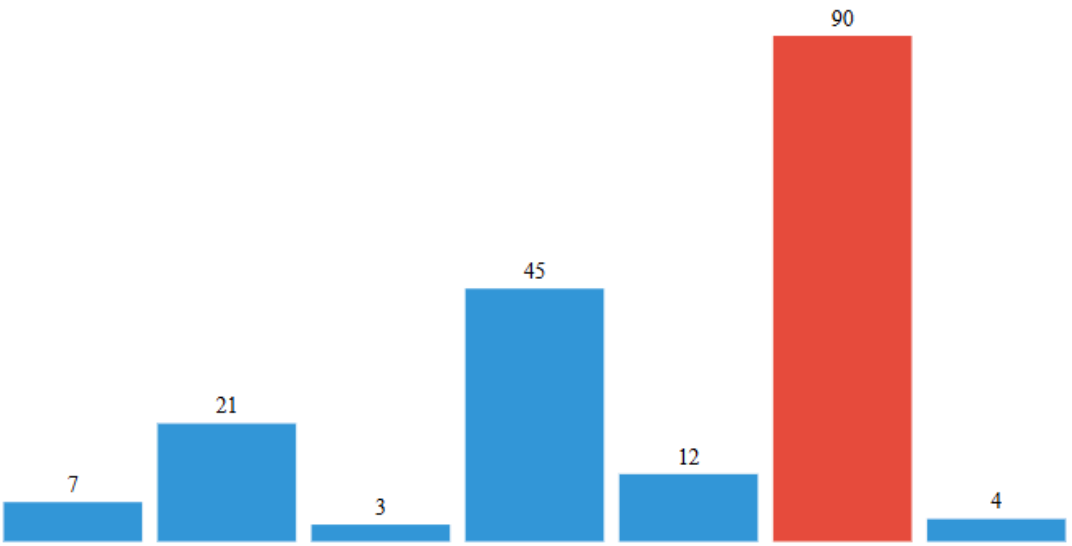
It should convert a number into an integer or Float (Number data type).

9. Part A – Dynamic Bar Chart

Quick test

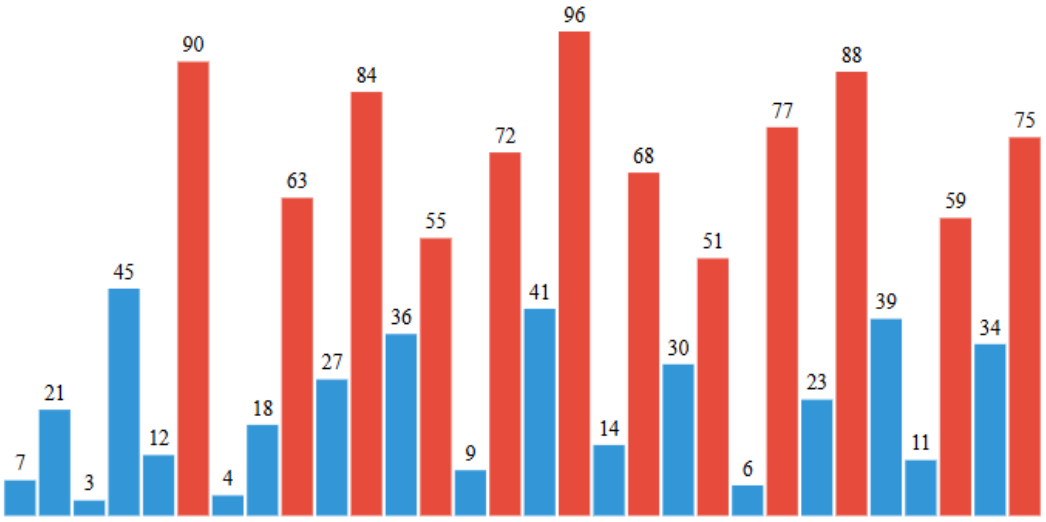
After your first version works, replace the dataset with:

[7, 21, 3, 45, 12, 90, 4]

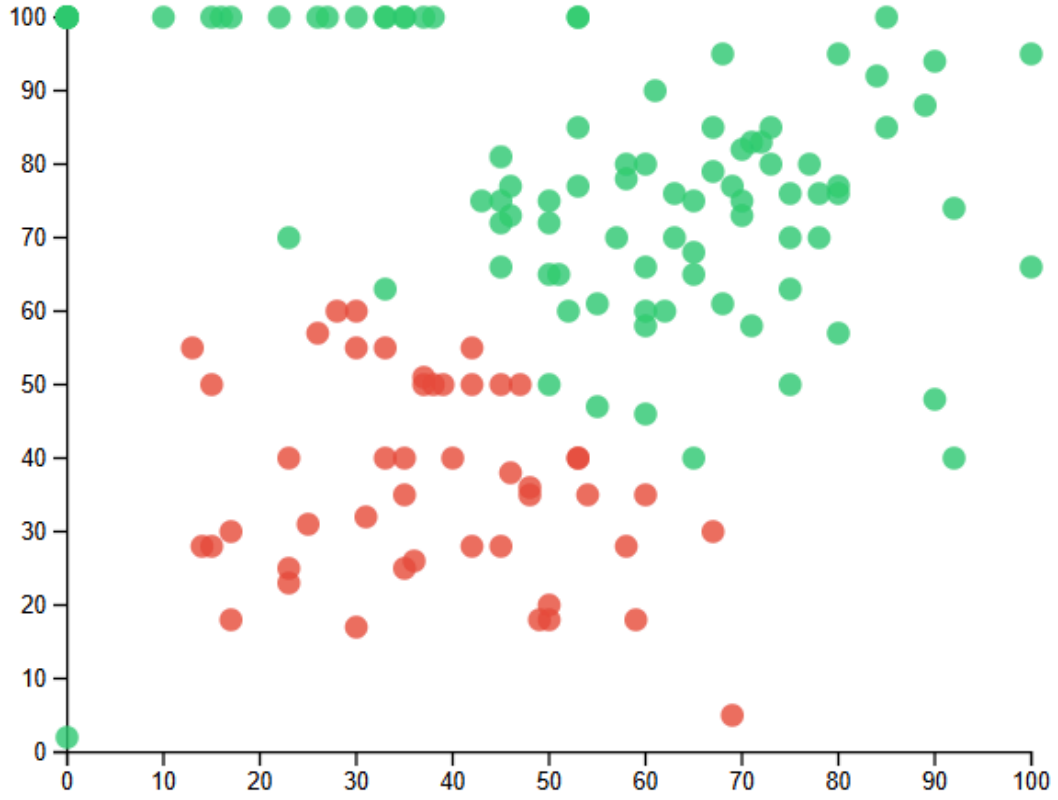


Then replace it with a larger dataset.

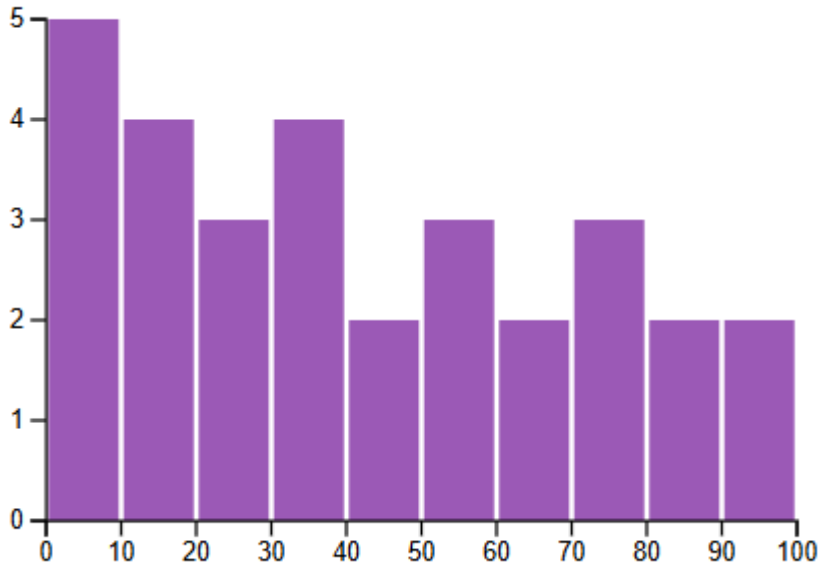
Does the chart still work? The chart still works



10. Part B – Scatterplot:



11. Part C — Histogram



1. What does one bin represent? Each bin represents the frequency (count) of data points falling within a specific numerical range or interval, such as 0–10, 10–20, up to 90–100.

2. Why is a histogram different from the original bar chart? The histogram looks like an overview for the first original one. The original just list out all the data instead grouping it

3. What happens when the data range changes?

- If the new data still ranges from 0 to 100, the bin widths remain equal, and the chart scales consistently.
- If the data has values above 100, those values are outside the histogram's domain and may not be included in the visible bins.
- For example, if the data range changes from 0–100 to 0–200, using the same 10 bins would make each bin wider, while using a fixed bin width of 10 would create more bins.

4. Why might the number of observations in each bin change?

The number of observations in each bin can change because:

The dataset values change

A value may move from one interval to another. For example, changing 29 to 35 moves it from the 20–30 bin to the 30–40 bin.

The number of data points changes

Adding more observations usually increases the counts in some bins.

The bin width changes

Wider bins contain more values, while narrower bins contain fewer values.

The data range changes

If the same number of bins is spread across a larger range, each bin represents a larger interval, so more observations may fall into each bin.

Values fall outside the domain

With the current 0–100 domain, values outside that range are not represented in the visible histogram.

The width of the bars is controlled by the bin intervals, while the height of the bars represents how many observations fall inside each interval. These are independent: the bars can have equal widths but different heights.

12. AI Collaboration Record

#	What we asked	What AI suggested	What we accepted/ changed	How we verified
1	Ask which steps we should follow to complete the task. Upload the file.	AI suggested that we start by creating files and how we can solve questions 9, 10, and 11.	We start by creating files and copying the code from the suggestions.	Open the Live Server and check manually if the code works.

2	Do not rewrite the code. Check the width of each bar in the bar chart. Identify and suggest a solution to make all bar fit in the page.	AI suggested to change the xScale and histogramGen to make sure the visualization is at its best.
		
		const xScale = d3.scaleLinear().domain([0,100]).range([0, innerW]);
		
		const histogramGen = d3.bin().domain(xScale.domain()).thresholds(d3.range(0, 101, 10));
			Accepted the suggestion.	Check the histogram to make sure it works.

15. VERIFY — AI Review

1. What did AI identify correctly?

- Falsy zero bug in CSV parsing: The use of +d.Midterm || +d.midterm treats a score of 0 as falsy, converting a legitimate zero score into NaN. This prevents the point from plotting correctly and causes the failing-grade check (courseScore < failThreshold) to fail silently.
- Missing Promise rejection handler: The d3.csv() call lacked a .catch() block to catch network errors, incorrect file paths, or CORS issues.
- Unfiltered invalid data binding: Rows containing null, empty strings, or non-numeric values are still bound to <circle> elements, causing orphaned or invisible nodes with NaN coordinates in the DOM.
- Fixed SVG layout discrepancy: The histogram's outer SVG width was hardcoded to 2000px while its inner chart width only used 300px, creating excessive empty whitespace.

2. What did AI miss?

- **D3 scales inverted coordinate mapping:** It did not flag whether negative values in the linear scale would cause SVG rendering issues due to SVG having its coordinate origin $(0, 0)$ at the top-left rather than bottom-left.
- **Responsive container resizing:** It did not address how the visualizations adapt when the browser window is resized (e.g., using viewBox instead of static pixel dimensions).
- **Handling duplicate or categorical values in the bar chart:** It focused primarily on numeric issues and missed verifying whether duplicate values would need distinct keys when migrating away from index-based mapping.

3. Did AI suggest anything irrelevant?

- **Complaint about different datasets (Point 9):** AI noted that the histogram and scatterplot used different datasets. This is irrelevant to code correctness because Part B explicitly requires external student CSV data, while Part C explicitly instructs using numerical data from the bar chart.

- **General-purpose support for negative numbers (Point 11):** Academic exam scores natively fall within non-negative domains (0 to 100 or 0 to 10), so building generalized negative-bar baseline logic is unnecessary over-engineering for this scenario.

4. Which suggestion did you verify?

- **The zero-score falsy bug (Issue 1):** Verified by passing a dummy test row { Midterm: "0", Final: "40" } through the parsing expression +d.Midterm || +d.midterm in the browser console. As predicted, +0 evaluates to 0 (falsy), causing the evaluation to fall through to +undefined which returns NaN.
- **Empty dataset handling in the histogram (Issue 8):** Tested drawHistogram([]) in the browser to verify if d3.max() returned undefined, creating degenerate scale domains.

5. What did you change

- **Fixed the CSV row conversion:** Replaced the falsy fallback operator || with explicit nullish checking or property existence checks (e.g., d.Midterm !== undefined ? +d.Midterm : +d.midterm) so a score of 0 is preserved as numeric 0.
- **Added error handling:** Chained .catch(err => console.error("Error loading CSV:", err)) to the d3.csv() Promise chain.
- **Filtered invalid records:** Added .filter(d => !isNaN(d.midterm) && !isNaN(d.final)) prior to data binding to prevent DOM pollution from NaN coordinates.
- **Corrected layout dimensions:** Adjusted the histogram SVG width and chart dimensions to match the consistent container sizing used across the page.

16. ADAPT

1. Which change was easier?

- **Change 2** (Scatterplot) was easier.

2. Why?

- Change 2 only required changing one number from 50 to 60. Change 1 required adding a new calculation (d3.mean(data)) to find the average first.

3. Which part of your original design made the change easy or difficult?

- **Easy**: Putting the color logic directly inside .attr("fill", ...) allowed editing the rule in one place without touching the scales or axes.
- **Difficult**: If the bar chart did not calculate summary values dynamically, we had to make sure the threshold was recalculated each time new data was passed.

4. What would you refactor if more visual rules were added later?

- Move color rules into separate helper functions (e.g., getColor(d)) instead of writing long if/else conditions inside the D3 chains.
- Calculate values like courseScore once in the data-loading step instead of recalculating them inside the .attr() call.

17. GROUP CHECK

C01:

D (datum) represent to the number or data which could be set to the attribute
e.g : dataset [10,20,30]
functions will run 3 times
1st: d=10 -> height =10
2nd: d=20 -> h=20
3rd: d=30 -> h=30

C02:

svg.selectAll("rect") selects nothing at first, because the SVG has no rects yet. After .data(dataset), D3 compares the data with the existing elements. Every data item with no matching element goes into the enter selection, which
is a set of placeholders for elements that should exist. .enter().append("rect") then creates one rect per placeholder. Without enter(), the selection is empty and nothing is drawn.

C03:

There is one element per data item for each append in the chain. For dataset = [12, 25, 8, 30, 17] with one rect per item, you get 5 rects. If the code also appends a text label per item, that adds 5 texts, so 10 elements. Add 1 for the <svg> itself and extra elements for any axes or groups. In the exam, count length(dataset) × number of append calls, then add the svg and any axis elements.

C04:

A common bug occurs when parsing numeric fields using fallback logic like +d.Midterm || +d.midterm; if a score is legitimately 0, it is treated as falsy and resolves to NaN, causing points with zero scores to vanish or miscalculate fail states.

C05:

Replace the mean calculation with median by updating the threshold rule from const threshold = d3.mean(data); to const threshold = d3.median(data);, keeping the visual encoding condition as .attr("fill", d => d > threshold ? "red" : "blue").

C06:

D3's CSV parser reads every table entry as a raw string; without explicit numeric conversion (e.g., using +d.value), operations like addition or weighted sums can fail or trigger string concatenation rather than numerical math.

C07:

Each student datum is bound to a <circle> where the quantitative midterm score is passed into xScale() to set cx, and the final score is passed into yScale() to set cy, transforming the raw values into an SVG x, y coordinate position.

C08:

One bin represents a continuous numerical interval x0, x1, and its visual bar height encodes the frequency - the total number of observations in the dataset whose values fall within that range.